

PlataformaOnline

Uma sala 3D com TV CRT LG e PlayStation 1 rodando jogos no navegador: como foi construída, como usar e como manter.



Sumário

Esta apostila foi escrita como um curso curto. Cada capítulo é uma "aula": começa explicando o porquê, mostra o como e termina com um resumo. No fim há receitas de manutenção, um guia de problemas e exercícios para fixar.

1. **Aula 1 — O que é o projeto** visão geral e o que acontece quando você liga a TV
2. **Aula 2 — Preparando o ambiente e rodando** pré-requisitos, comandos, onde colocar jogos e BIOS
3. **Aula 3 — A arquitetura** servidor, página, cena 3D, emulador e como eles conversam
4. **Aula 4 — Tour pelos arquivos** o papel de cada arquivo, um por um
5. **Aula 5 — Estudo de caso 1: o jogo .cue + .bin que não iniciava**
6. **Aula 6 — Estudo de caso 2: o controle que o emulador não reconhecia**
7. **Aula 7 — Estudo de caso 3: luzes e sombras para dar realismo**
8. **Aula 8 — Receitas de manutenção** como alterar cada parte com segurança
9. **Aula 9 — Diagnóstico de problemas** sintoma → causa provável → o que fazer
10. **Aula 10 — Exercícios e glossário**

Como estudar esta apostila

Leia as aulas 1 a 4 em ordem: elas dão o mapa mental do projeto. As aulas 5 a 7 contam o que foi corrigido e adicionado, e por quê. As aulas 8 e 9 são para consulta: volte a elas quando for mexer no projeto.

Regras do projeto (não quebre!)

- Use **só os SDKs .NET já instalados**. O projeto está fixado no **.NET 8.0.302** pelo `global.json`. Não instale SDKs, runtimes ou workloads novos.
- O `.csproj` fica **direto na raiz** (`D:\DevGames\PlataformaOnline`), sem subpastas de solução.
- **Não adicione pacotes NuGet** além dos que vêm no template Blazor do .NET 8 sem combinar antes.

1 O que é o projeto

Imagine uma sala dos anos 2000: um rack de madeira, uma TV de tubo LG prata e um PlayStation 1 na prateleira. É isso que o projeto desenha em 3D no navegador, e os jogos funcionam de verdade.

A experiência do usuário, passo a passo

1. A página abre mostrando a sala em 3D. Dá para girar a câmera arrastando o mouse.
2. Você aperta **POWER** (botão da barra, tecla `P` ou clique no botão da TV/PS1). A tela do CRT "acende" com aquela linha horizontal que se abre.
3. Aparece o OSD azul **AV1** da TV, depois o boot do PlayStation (tela branca da Sony e o logo PS), com som.
4. Surge o **menu de jogos**, com tudo que está na pasta `Jogos/`.
5. Ao escolher um jogo, a câmera faz zoom até enquadrar a tela. Um **iframe** com o emulador é posicionado exatamente por cima da tela da TV.
6. O jogo roda pelo **EmulatorJS**. `Esc` volta ao menu; `F` ou **Select + Start** alterna a tela cheia.

As tecnologias, em uma frase cada

Tecnologia	Para que serve aqui
ASP.NET Core / Blazor (.NET 8)	O servidor. Entrega a página, lista os jogos (<code>/api/games</code>), serve os arquivos dos jogos e da BIOS e empacota <code>.cue + .bin</code> em <code>.zip</code> .
Three.js 0.160	A biblioteca de 3D no navegador (WebGL). Desenha a sala, a TV, o PS1, as luzes e as sombras.
Canvas 2D	Desenha o conteúdo da tela da TV (boot, menu), que vira uma textura no modelo 3D.
Shader GLSL	Dá o visual de CRT na tela: curvatura, scanlines, chiado e a animação de ligar.
EmulatorJS	Emulador de PS1 (RetroArch compilado para WebAssembly) que roda dentro do <code>player.html</code> .
Gamepad API	API do navegador que lê joysticks USB/Bluetooth.

Resumo da aula

O servidor .NET é simples: lista e entrega arquivos. Quase toda a "mágica" visual está no JavaScript da pasta `wwwroot/js`, e o jogo em si roda no EmulatorJS dentro de um `iframe`.

2 Preparando o ambiente e rodando

Antes de mexer em qualquer coisa, você precisa saber rodar o projeto. Vamos do zero.

Pré-requisitos

- **.NET SDK 8.0.302** (já instalado na máquina). Confira com `dotnet --list-sdks`.
- Um navegador moderno: **Chrome** ou **Edge** (recomendados para joysticks).
- **Internet** na primeira execução: o Three.js e o EmulatorJS vêm de CDN. Para rodar offline, veja a Receita 8.4.

Rodando

```
# na pasta do projeto
cd D:\DevGames\PlataformaOnline
dotnet build      # compila e mostra erros, se houver
dotnet run        # sobe o servidor (http://localhost:5136 pelo perfil "http")
```

Abra o endereço que aparecer no terminal. Para parar o servidor, use `Ctrl` + `C` no terminal.

Dica do professor

Mudou só arquivos de `wwwroot/` (JS, CSS, HTML)? Em geral basta recarregar a página com `Ctrl` + `F5`.
Mudou arquivos `.cs` ou `.razor`? Pare e rode `dotnet run` de novo.

Colocando jogos

Copie seus jogos (dumps dos seus próprios discos) para a pasta `Jogos/`. Formatos aceitos:

Formato	Observação
<code>.chd</code> , <code>.pbp</code>	Recomendados. Um arquivo só, compactado.
<code>.cue</code> + <code>.bin</code>	Funciona. O servidor junta os dois em um <code>.zip</code> automaticamente (veja a Aula 5). O <code>.bin</code> deve ter o nome citado dentro do <code>.cue</code> .
<code>.iso</code> , <code>.img</code> , <code>.bin</code>	Imagem única do disco.
<code>.zip</code> , <code>.7z</code>	Com o <code>.cue</code> e os <code>.bin</code> dentro.

Capa opcional: coloque uma imagem `.png`, `.jpg`, `.jpeg` ou `.webp` com o **mesmo nome** do jogo.

Exemplo: `Crash.chd` e `Crash.png`.

Nome no menu: é o nome do arquivo sem extensão, com `_` trocado por espaço.

BIOS (opcional)

Coloque a BIOS do seu console em `Bios/`. O servidor procura nesta ordem: `scph5501.bin`, `scph5500.bin`, `scph5502.bin`, `scph1001.bin`, `scph7001.bin`, `scph101.bin` e, por fim, qualquer `.bin`. Sem BIOS, o emulador usa uma BIOS simulada (HLE), que funciona na maioria dos jogos.

Comandos do usuário

Ação	Teclado	Joystick	Mouse
Ligar/desligar	P	Start/X com a TV desligada	POWER, ou botão da TV/PS1
Navegar no menu	Setas	D-pad ou analógico	Clique no jogo
Iniciar jogo	Enter / Espaço	X ou Start	Clique no jogo selecionado
Voltar ao menu	Esc / Backspace	— (ver Exercício 5)	Botão MENU
Tela cheia	F	Select + Start	Botão TELA CHEIA

3 A arquitetura

Arquitetura é "quem faz o quê e como as partes conversam". Entendendo o desenho abaixo, você entende o projeto.

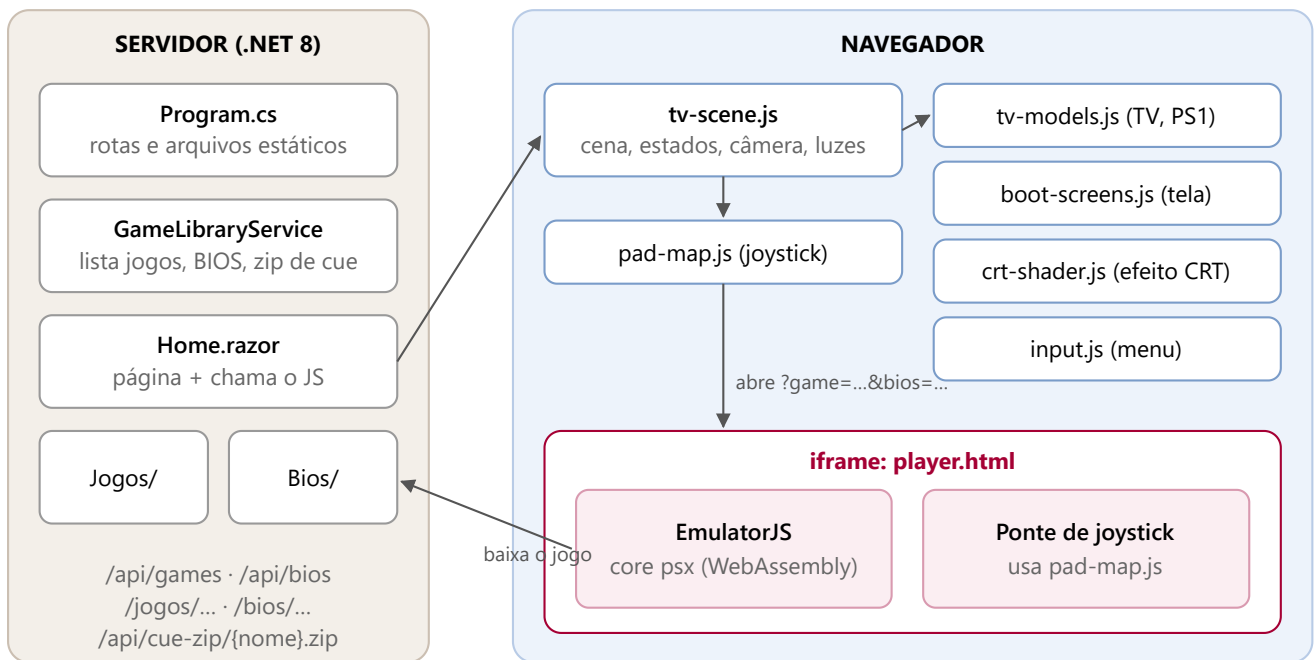
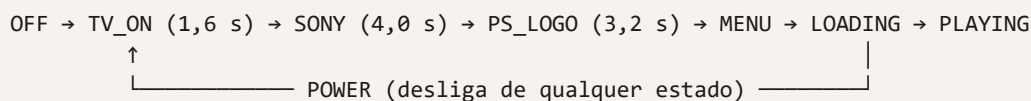


Figura 1: o servidor entrega a página e os arquivos; o navegador faz a cena 3D e roda o emulador num iframe.

O caminho de um clique até o jogo rodar

1. **Home.razor** renderiza a página e, no primeiro render, importa `tv-scene.js` e chama `init(stage, { games, biosUrl, dataPath })`. A lista de jogos vem do `GameLibraryService`.
2. **tv-scene.js** monta a cena (render, câmera, luzes, sala, TV, PS1) e começa o loop de animação (`_loop`, cerca de 60 vezes por segundo).
3. O loop é uma **máquina de estados**:



Em cada estado, o `ScreenPainter` (`boot-screens.js`) desenha a tela correspondente num canvas 640×480. Esse canvas vira textura, e o shader CRT a exibe na TV.

4. No **MENU**, ao confirmar, a câmera faz zoom até a tela (`startGame`). Ao terminar o movimento, `_openEmulator` carrega `player.html?game=...&bios=...&title=...&data=...` no iframe.
5. **player.html** configura as variáveis `EJS_*` e carrega o `loader.js` do `EmulatorJS`, que baixa o jogo pela URL recebida e inicia o core psx.
6. O iframe é reposicionado a cada frame (`_positionEmulator`) para ficar **exatamente** em cima da tela 3D, projetando os cantos da tela para coordenadas de pixel.
7. Iframe e página conversam por `postMessage: ready, back (Esc) e fullscreen`.

Por que um iframe?

O EmulatorJS espera ser dono da página: ele cria a própria interface, captura teclado e usa variáveis globais (`window.EJS_*`). Isolando-o num iframe, a cena 3D não sofre interferência, e "fechar o jogo" é só trocar o `src` do iframe para `about:blank`.

Configuração (`appsettings.json`)

```
{
  "GamesFolder": "Jogos",           // pasta dos jogos (relativa ao projeto ou absoluta)
  "BiosFolder": "Bios",             // pasta da BIOS
  "EmulatorJsDataPath": "https://cdn.emulatorjs.org/stable/data/" // de onde vem o EmulatorJS
}
```

4 Tour pelos arquivos

Um mapa do projeto. Quando surgir a dúvida "onde eu mexo para...?", comece por esta tabela.

Arquivo	Responsabilidade
PlataformaOnline.csproj	Projeto .NET (net8.0 , sem pacotes extras).
global.json	Fixa o SDK 8.0.302.
appsettings.json	Pastas de jogos/BIOS e endereço do EmulatorJS.
Program.cs	Registra serviços; serve Jogos/ em /jogos e Bios/ em /bios ; APIs /api/games , /api/bios e /api/cue-zip/{nome}.zip .
Services/GameLibraryService.cs	Varre a pasta de jogos, acha capas, escolhe a BIOS e gera o zip de .cue .
Models/GameInfo.cs	O "formulário" de cada jogo: Id , Title , FileName , Url , CoverUrl , SizeBytes .
Components/App.razor	HTML base; o importmap que diz de onde vem o Three.js (versão 0.160.0).
Components/Pages/Home.razor	A página: estrutura (cena, iframe, barra de botões) e a chamada ao tv-scene.js .
wwwroot/css/tv.css	Visual da barra estilo controle remoto, da dica e do iframe.
wwwroot/js/tv-scene.js	O coração. Renderer, câmera, luzes, sombras, sala, abajur, estados, cliques, áudio sintetizado e posicionamento do emulador.
wwwroot/js/tv-models.js	Modelagem procedural da TV LG Flatron e do PS1 (sem arquivos 3D externos).
wwwroot/js/boot-screens.js	Desenha no canvas o OSD da TV, o boot Sony/PS, o menu e a tela de carregamento.
wwwroot/js/crt-shader.js	Shader do CRT: curvatura, scanlines, ruído, vinheta e animação de ligar.
wwwroot/js/input.js	Teclado + joystick para navegar fora do jogo.
wwwroot/js/pad-map.js	Novo. Lê qualquer joystick e o converte para o layout padrão.
wwwroot/player.html	Página do emulador (no iframe) e a ponte de joystick .
Jogos/ , Bios/	Seus arquivos. Não fazem parte do código.
DOCS/	Esta apostila (HTML editável + PDF).

Padrão de modelagem: "procedural"

Repare que não há arquivos .glb ou .obj . TV, PS1, rack e abajur são montados com formas básicas do Three.js (caixas, cilindros, extrusões), e logotipos e texturas são desenhados em canvas. Vantagem: nenhum asset proprietário e tudo editável por código. Para mudar o tamanho da TV, por exemplo, altere W , H , screenW e screenH em createLgTv() .

5 Estudo de caso 1: o jogo .cue + .bin que não iniciava

O primeiro problema da sessão: "adicionei um jogo, mas não iniciou". Vamos ver como um bug é investigado de verdade.

O sintoma

A pasta `Jogos/` tinha `PslHelloWorld0.1.cue` (146 bytes) e `PslHelloWorld0.1.bin` (13 MB). O jogo aparecia no menu, mas não rodava.

A investigação

Um arquivo `.cue` é só um **índice em texto**. Ele diz "os dados estão no arquivo tal":

```
FILE "PslHelloWorld0.1.bin" BINARY
  TRACK 01 MODE2/2352
    INDEX 01 00:00:00
  TRACK 02 AUDIO
    ...
```

O serviço listava o `.cue` (e escondia o `.bin` do menu), e a URL do jogo era `/jogos/PslHelloWorld0.1.cue`. Só que o EmulatorJS baixa **uma única URL**. Ele recebia o índice de 146 bytes e não tinha como buscar o `.bin`. Era como receber o sumário de um livro sem o livro.

A solução

O EmulatorJS aceita `.zip` e procura o `.cue` lá dentro. Então o servidor passou a **empacotar** o `.cue` com todos os arquivos que ele cita:

1. Em `ToGameInfo`, se o jogo é `.cue`, a URL vira `/api/cue-zip/{nome}.zip` em vez de `/jogos/{nome}.cue`.
2. `GetCueZip(baseName)` lê as linhas `FILE "..."` do `.cue` (expressão regular), junta os arquivos existentes e cria um zip **sem compressão** (`CompressionLevel.NoCompression`). Sem compressão porque é bem mais rápido e as imagens de disco quase não comprimem.
3. O zip vai para um **cache** em `%TEMP%\PlataformaOnline\cue-zip\` e só é refeito se o `.cue` ou um `.bin` for mais novo que o zip.
4. A rota em `Program.cs` entrega o arquivo com suporte a *range requests* (download em partes).

`Services/GameLibraryService.cs` (trecho)

```
var isCue = Path.GetExtension(path).Equals(".cue", StringComparison.OrdinalIgnoreCase);
...
Url: isCue
    ? "/api/cue-zip/" + Uri.EscapeDataString(baseName) + ".zip"
    : "/jogos/" + Uri.EscapeDataString(fileName),
```

`Program.cs` (trecho)

```
app.MapGet("/api/cue-zip/{name}.zip", (string name, GameLibraryService lib) =>
    lib.GetCueZip(name) is { } zip
        ? Results.File(zip, "application/zip", name + ".zip", enableRangeProcessing: true)
        : Results.NotFound());
```

Detalhes de segurança e robustez

- O nome recebido na URL é validado (`Path.GetInvalidFileNameChars`) e só se usa `Path.GetFileName` dos arquivos citados no `.cue`. Assim ninguém consegue pedir arquivos fora da pasta `Jogos/` com `..\.`
- O zip é escrito primeiro num `.tmp` e depois renomeado, e um `lock` evita dois zips sendo criados ao mesmo tempo.

Limitação conhecida

O tamanho exibido para jogos `.cue` (`SizeBytes`) é o do próprio `.cue`, não o do jogo inteiro. É só informativo. Veja o Exercício 2.

Lição

Antes de mexer no código, entenda o **formato dos dados** e o **contrato** de cada peça. O EmulatorJS espera "uma URL = um jogo completo", e nossa URL apontava para só um pedaço do jogo.

6 Estudo de caso 2: o controle que o emulador não reconhecia

Segundo problema: "o controle não foi identificado pelo emulador". Esse é mais sutil e ensina muito sobre a Gamepad API.

Como o navegador enxerga um joystick

`navigator.getGamepads()` devolve uma lista de objetos com `buttons[]` (pressionado ou não), `axes[]` (valores de -1 a 1) e `mapping`.

- Se `mapping === "standard"`, o navegador reorganizou o controle num layout conhecido: 0 = X, 1 = O, 2 = □, 3 = △, 12–15 = direcional.
- Se `mapping === ""` (controle "genérico"), os botões vêm **na ordem do fabricante**. Em adaptadores e controles USB baratos de PS1/PS2, é comum: 0 = △, 1 = O, 2 = X, 3 = □. O direcional chega por um **hat switch** (geralmente `axes[9]`) ou pelos eixos 0/1.
- Por privacidade, o Chrome só revela o joystick depois que **um botão é apertado** com a página em foco.

A causa

Lendo o código-fonte do EmulatorJS (`gamepad.js` e `emulator.js`), vimos que o mapeamento padrão dele procura `DPAD_UP` = botão 12, `BUTTON_1` = botão 0 etc., ou seja, o layout *standard*. Em um controle genérico, o botão 12 não existe e o direcional chega por eixo. Resultado: o jogo não reage.

A solução em três partes

1. `pad-map.js`: um "tradutor" único de joystick

A função `readPad(pad)` devolve sempre `{ buttons: [16 booleanos no layout standard], lx, ly, rx, ry }`:

```
const GENERIC_FACE = [2, 1, 3, 0]; // standard [X, O, □, △] ← botão no controle genérico

if (pad.mapping === 'standard') { /* copia direto */ }
else {
  GENERIC_FACE.forEach((src, dst) => { b[dst] = raw(src); });
  // hat switch: -1 = cima, avança 2/7 por posição no sentido horário; repouso fica fora de [-1,
1]
  const dir = Math.round((hat + 1) * 3.5); // 0 = cima ... 7 = cima-esquerda
}
// o analógico esquerdo também aciona o direcional
if (ly < -0.5) b[12] = true; ...
```

2. Ponte de joystick no `player.html`

Em vez de depender do leitor interno do EmulatorJS, um script (`<script type="module">`) roda a cada frame:

- Pega os controles do próprio iframe. Se o iframe não enxergar nenhum, usa os que a página da TV vê (`parent.navigator`, permitido porque é a mesma origem).

- Quando o jogo começa (`EJS_emulator.started`), desliga o leitor interno (`emu.gamepad.terminate()`) para não haver comandos dobrados.
- Converte com `readPad` e envia ao core com `gameManager.simulateInput(jogador, id, valor)` , **só quando o valor muda**.

A tabela `RETRO` traduz o índice *standard* para o ID que o RetroArch (dentro do EmulatorJS) entende:

Botão (standard)	0 ✕	1 ○	2 □	3 △	4 L1	5 R1	6 L2	7 R2
ID RetroArch	0 (B)	8 (A)	1 (Y)	9 (X)	10	11	12	13
Botão (standard)	8 Select	9 Start	10 L3	11 R3	12 ↑	13 ↓	14 ←	15 →
ID RetroArch	2	3	14	15	4	5	6	7

Analógicos: IDs 16/17 (esquerdo X +/-), 18/19 (esquerdo Y +/-), 20–23 (direito), com valores de 0 a 32767 (0x7fff). Até 4 jogadores, na ordem em que os controles aparecem.

3. Menu da TV mais esperto (`input.js`)

- Usa o mesmo `readPad` , então controles genéricos também navegam no menu.
- Confere a lista de controles **a cada frame** em vez de esperar o evento `gamepadconnected` , que nem sempre dispara (por exemplo, depois de recarregar a página).
- A barra mostra "sem controle (aperte um botão)" e, ao passar o mouse, o nome e o tipo de cada controle (`standard` ou `genérico`).

Como foi testado

Com o jogo rodando, simulamos um controle genérico substituindo `navigator.getGamepads` no iframe. Apertar o botão 2 (✕ genérico) gerou o ID 0 (✕ do PS1); hat para cima gerou o ID 4 (↑). Soltar os dois enviou 0 para ambos.

Consequência a conhecer

Como a ponte assume o joystick, o remapeamento **de joystick** feito na tela de controles do EmulatorJS deixa de valer. O teclado continua configurável por lá. Para remapear o joystick, edite o código (Receita 8.5).

7 Estudo de caso 3: luzes e sombras para dar realismo

A terceira tarefa foi estética: a sala precisava parecer de verdade. Realismo em 3D vem principalmente de **luz motivada** (toda luz tem uma fonte visível ou plausível) e de **sombras** que "assentam" os objetos no chão.

Conceitos rápidos de iluminação no Three.js

Tipo de luz	Como se comporta	Usada aqui como
<code>HemisphereLight</code>	Luz ambiente suave: uma cor vinda de cima e outra de baixo. Não faz sombra.	Preenchimento bem fraco (0,18)
<code>PointLight</code>	Lâmpada que ilumina em todas as direções. Sombra em 6 direções (mais cara).	Abajur e brilho pontual da tela
<code>DirectionalLight</code>	Raios paralelos, como o sol ou a lua. Sombra barata e nítida.	Luar azulado pela "janela"
<code>RectAreaLight</code>	Painel retangular que emite luz suave. Não faz sombra.	A superfície da tela da TV

O que foi montado (`tv-scene.js` → `_buildScene`)

1. **Abajur de chão** (`_buildFloorLamp`) no canto esquerdo: base, haste, cúpula de tecido translúcida (material com `emissive`, que brilha sozinho) e lâmpada. Dentro dela, uma `PointLight` quente (`0xffb46b`, intensidade 2,4) que **projeta sombras**. A cúpula não projeta sombra, senão bloquearia a própria luz.
2. **Luar**: `DirectionalLight` azulada (`0x8fa6ff`, 0,55) vinda da direita e do alto, com sombras longas na parede. O contraste quente × frio é um truque clássico de fotografia.
3. **Luz da tela**: um `RectAreaLight` do tamanho da tela, apontado para a frente, mais uma `PointLight` com sombra. A cor vem da **imagem atual**: a cada 4 frames o canvas da tela é reduzido para 8×6 pixels e a média das cores é calculada (`_updateScreenGlow`). Uma leve tremulação imita o CRT. Durante o jogo, a imagem está no iframe e não pode ser lida, então usamos um azulado que varia devagar.
4. **Sombras em tudo que é sólido**: TV, PS1, cabo e rack recebem `castShadow` e `receiveShadow` via `traverse`. Tela, vidro e peças transparentes ficam de fora.
5. **Ambiente**: textura de reboco na parede, rodapé e um tapete. Superfícies com textura deixam a luz e a sombra mais "legíveis".

`tv-scene.js`: configurando uma sombra (exemplo do abajur)

```
this.lampLight = new THREE.PointLight(0xffb46b, 2.4, 0, 2); // cor, intensidade, alcance (0 = infinito), decaimento físico
this.lampLight.castShadow = true;
this.lampLight.shadow.mapSize.set(1024, 1024); // resolução da sombra (maior = mais nítida e mais pesada)
this.lampLight.shadow.bias = -0.0015; // evita "listras" (shadow acne)
this.lampLight.shadow.normalBias = 0.02; // idem, em superfícies inclinadas
```

Vocabulário: shadow acne e peter-panning

Shadow acne são listras ou manchas escuras quando a superfície "faz sombra em si mesma". Corrige-se com `bias` negativo ou `normalBias` positivo. Exagerar causa **peter-panning**: a sombra "descola" do objeto, que parece flutuar. Ajuste em passos pequenos.

Verificação

A cena foi aberta no Chrome, ligada e levada até o jogo, sem erros no console. A primeira versão deixava a cúpula estourada e grande demais em primeiro plano; o abajur foi afastado para o canto (`-1.2, 0, -0.38`) e o brilho da cúpula reduzido (`emissiveIntensity 0,35`).

8 Receitas de manutenção

Receitas curtas para as alterações mais prováveis. Siga o passo a passo e teste ao final.

8.1 Aceitar um novo formato de jogo

1. Em `GameLibraryService.cs`, adicione a extensão em `GameExtensions`.
2. Confirme que o EmulatorJS (core `psx`) aceita o formato.
3. `dotnet run` e confira em `/api/games` se o jogo aparece.

8.2 Mudar as pastas de jogos ou BIOS

Edite `GamesFolder/BiosFolder` no `appsettings.json`. Caminhos absolutos funcionam (ex.:

`"E:\\Roms\\PS1"`, com barra dupla no JSON). Reinicie o servidor.

8.3 Mudar tempos do boot, câmera e menu

- Durações: `DURATION` no topo de `tv-scene.js` (`TV_ON`, `SONY`, `PS_LOGO`, `LOADING`).
- Posição inicial da câmera: `ROOM_CAMERA`. Limites de giro e zoom: `this.controls.min/max...` em `_buildRenderer`.
- Visual do menu: `menu()` e a constante `MENU` em `boot-screens.js`.

8.4 Rodar sem internet

1. Baixe a release do EmulatorJS e extraia em `wwwroot/emulatorjs/`.
2. Em `appsettings.json`: `"EmulatorJsDataPath": "/emulatorjs/data/"`.
3. Baixe `three.module.js` e a pasta `examples/jsm` do Three.js 0.160.0 para `wwwroot/lib/three/` e ajuste o `importmap` em `App.razor`.

8.5 Trocar o mapeamento de botões do joystick

- **Controle genérico com botões trocados:** ajuste `GENERIC_FACE` em `pad-map.js`. Cada posição é [`X`, `O`, `□`, `△`] e o valor é o número do botão no seu controle. Para descobrir os números, abra o console (`F12`), aperte um botão e rode:

```
navigator.getGamepads()[0].buttons.map((b, i) => b.pressed ? i : null).filter(i => i !== null)
```

- **Mudar o que cada botão faz no jogo:** altere a tabela `RETRO` no `player.html` (veja a tabela da Aula 6).
- **Mudar os botões do menu da TV:** constante `BTN` em `input.js`.

8.6 Ajustar luzes e sombras

Quero...	Onde mexer (tv-scene.js)
Sala mais clara ou escura	<code>r.toneMappingExposure (0,9)</code> em <code>_buildRenderer</code> ; intensidade da <code>HemisphereLight (0,18)</code> .
Abajur mais forte ou mais quente	<code>new THREE.PointLight(0xffb46b, 2.4, ...)</code> : cor e intensidade.
Mudar o abajur de lugar	<code>lampPos (x, y, z)</code> . A luz acompanha automaticamente.
Luar mais ou menos visível	<code>new THREE.DirectionalLight(0x8fa6ff, 0.55)</code> e <code>moon.position</code> .
Tela iluminando mais a sala	Em <code>_updateScreenGlow</code> : <code>level * 5</code> (área) e <code>level * 0.9</code> (pontual).
Sombras mais nítidas	Aumente <code>shadow.mapSize</code> (ex.: 2048). Custa desempenho.
Melhorar o desempenho	Desligue a sombra da luz da tela (<code>this.screenLight.castShadow = false</code>), reduza <code>mapSize</code> ou, em último caso, <code>r.shadowMap.enabled = false</code> .

8.7 Adicionar um objeto à sala (com sombra)

```
const vaso = new THREE.Mesh(
  new THREE.CylinderGeometry(0.06, 0.05, 0.2, 24),
  new THREE.MeshStandardMaterial({ color: 0x2f5d50, roughness: 0.4 }));
vaso.position.set(0.45, 0.6 + 0.1, 0.1); // em cima do rack (tampo em y = 0.6)
vaso.castShadow = true;
vaso.receiveShadow = true;
this.scene.add(vaso); // coloque no fim de _buildRoom()
```

Unidades em **metros**; o eixo Y é a altura; a parede fica em `z = -0.75`. Use `MeshStandardMaterial` (reage a todas as luzes, inclusive a `RectAreaLight`).

8.8 Atualizar o Three.js

Troque a versão nas duas linhas do `importmap` em `App.razor`. **Atenção:** entre versões, nomes de addons e o cálculo de intensidade das luzes podem mudar. Teste a cena inteira (ligar, boot, menu, jogo) depois de atualizar.

8.9 Checklist depois de qualquer mudança

1. `dotnet build` sem erros.
2. Abrir a página com o console aberto (`F12`) e verificar se não há erros vermelhos.
3. Ligar a TV, ver o boot, navegar no menu com teclado e joystick.
4. Abrir um jogo `.chd` e um `.cue`; testar Esc, tela cheia e Select + Start.

9 Diagnóstico de problemas

Quando algo der errado, siga esta tabela antes de alterar código. A maioria dos problemas está em arquivos ou no navegador, não no projeto.

Sintoma	Causa provável	O que fazer
Jogo não aparece no menu	Extensão não suportada; arquivo em subpasta.	Coloque o arquivo direto em <code>Jogos/</code> ; confira <code>/api/games</code> ; veja a Receita 8.1.
Jogo aparece mas não inicia	Arquivos <code>.bin</code> faltando ou com nome diferente do citado no <code>.cue</code> .	Abra o <code>.cue</code> no Bloco de Notas e confira os nomes nas linhas <code>FILE</code> . Teste <code>/api/cue-zip/NOME.zip</code> no navegador.
Jogo antigo <code>.cue</code> continua com erro após trocar o <code>.bin</code>	Zip antigo no cache.	Apague <code>%TEMP%\PlataformaOnline\cue-zip\</code> .
"Não foi possível carregar o EmulatorJS"	Sem internet ou CDN fora do ar.	Verifique a conexão ou use a Receita 8.4.
Barra mostra "sem controle"	O navegador só revela o joystick após um botão ser apertado.	Clique na página e aperte um botão do controle. Teste em https://hardwaretester.com/gamepad .
Controle detectado, mas botões trocados	Controle genérico com outra ordem de botões.	Receita 8.5 (<code>GENERIC_FACE</code>).
Direcional não funciona no jogo	Hat do controle num eixo diferente do 9.	No console, observe <code>navigator.getGamepads()[0].axes</code> enquanto aperta o direcional e ajuste o índice em <code>readPad</code> .
Controle não funciona via IP da rede (ex.: <code>http://192.168...</code>)	A Gamepad API exige contexto seguro (HTTPS ou localhost).	Use <code>localhost</code> ou o perfil <code>https</code> (<code>dotnet run --launch-profile https</code>).
Cena lenta ou travando	Sombras e resolução altas para a placa de vídeo.	Receita 8.6, linha "Melhorar o desempenho".
Listras escuras nas superfícies	Shadow acne.	Ajuste <code>bias / normalBias</code> da luz responsável (Aula 7).
Iframe do jogo "desalinhado" da tela	Mudança no tamanho da tela da TV sem atualizar <code>screenSize</code> .	Em <code>createLgTv()</code> , mantenha <code>screenSize</code> igual a <code>screenW / screenH</code> .
<code>dotnet</code> reclama do SDK	<code>global.json</code> pede 8.0.302.	Não instale outro SDK; confira com <code>dotnet -list-sdks</code> .

Método geral de depuração

1. **Reproduza** o problema de forma confiável.
2. **Isole** a camada: servidor (abra a URL da API direto), página (console F12) ou emulador (console do iframe).
3. **Leia** a mensagem de erro inteira e o código da peça envolvida, inclusive de bibliotecas, como fizemos com o EmulatorJS.
4. **Mude uma coisa por vez** e teste de novo.

10 Exercícios e glossário

Aprender é fazer. Os exercícios vão do mais simples ao mais desafiador e, juntos, passam por todas as camadas do projeto.

Exercícios

1. **Aquecimento:** mude a cor do LED do PS1 de verde para azul. *Dica:* `ledMat` em `createPlayStation()`.
2. **Servidor:** faça `SizeBytes` de jogos `.cue` somar o tamanho de todos os `.bin` citados. *Dica:* reaproveite a leitura das linhas `FILE` de `GetCueZip` extraíndo-a para um método.
3. **Iluminação:** faça a cúpula do abajur apagar ou acender com a tecla `L`, mexendo em `lampLight.intensity` e `lampShadeMat.emissiveIntensity`.
4. **Menu:** mostre no menu o tamanho de cada jogo em MB. *Dica:* `sizeBytes` já chega no JavaScript.
5. **Joystick:** faça a combinação `Select + O` do controle sair do jogo e voltar ao menu (hoje só o `Esc` e o botão `MENU` fazem isso). *Dica:* na ponte do `player.html`, envie `parent.postMessage({ source: 'ps1-player', type: 'back' }, location.origin)` ao detectar `Select + O`.
6. **Desafio:** adicione uma janela 3D na parede direita, de onde o luar "entra", e posicione a `DirectionalLight` alinhada a ela.

Glossário

Termo	Significado
Blazor (InteractiveServer)	Framework .NET para páginas web. Neste modo, a lógica do componente roda no servidor e conversa com o navegador por uma conexão em tempo real.
BIOS / HLE	O "sistema" do console. HLE (<i>High-Level Emulation</i>) é uma BIOS imitada pelo emulador.
.cue / .bin	Índice em texto do disco / dados brutos das trilhas.
.chd	Formato compactado de imagens de disco (MAME). Um arquivo por jogo.
CDN	Servidor público que entrega bibliotecas (Three.js, EmulatorJS).
iframe	Uma página dentro de outra página.
postMessage	Forma segura de duas janelas (página e iframe) trocarem mensagens.
Gamepad API	API do navegador para ler joysticks.
Mapping "standard"	Layout de botões padronizado pelo navegador; o oposto é o "genérico".
Hat switch	Direcional que o controle informa como um único eixo com 8 posições.
RetroArch / core	Base do EmulatorJS; o core <code>psx</code> é o emulador de PlayStation.
Shader	Pequeno programa que roda na placa de vídeo para colorir pixels.
Shadow map	Técnica de sombra: a cena é renderizada do ponto de vista da luz para saber o que está "escondido".
Emissive	Propriedade de material que faz o objeto parecer emitir luz própria.
Tone mapping	Conversão das cores calculadas para a faixa que o monitor exibe (aqui, ACES Filmic).
Range request	Download de um arquivo em partes, útil para arquivos grandes.

Palavra final do professor

Este projeto tem uma divisão clara de responsabilidades: o **servidor** cuida dos arquivos, a **cena** cuida do visual e o **emulador** cuida do jogo. Quando for alterar algo, pergunte primeiro: "em qual dessas camadas está o que eu quero mudar?". Na maioria das vezes, a resposta leva direto ao arquivo certo.